

# Unit 3: Working with Web Framework 8 hours

**Compiled By**

Lecturer, Sunil Bahadur Bist

NAST, Dhangadhi

# Contents

- Framework overview:
- Introduction to Flask Framework
- Setting up a Flask environment
- Flask basics: Routes, Views, and Templates
- Handling HTTP requests and responses
- Building Basic Web Functionality
  - Implementing form submission and input validation
  - Understanding and managing cookies and sessions
- Flask redirect, Message flashing , File upload
- Sending Email

# Framework overview

- **Frameworks** build upon foundational web concepts and introduce more advanced techniques for creating dynamic and complex web applications.
- They provide a structured approach to development, simplifying tasks like building user interfaces, managing data, and handling server-side logic.
- A **web framework** is a software framework designed to aid the development of web applications including web services, web resources, and web APIs.
- It provides a standard way to **build** and **deploy** web apps.

# Framework overview

## Examples:

- **Python:** Flask, Django
- **JavaScript:** Express.js (Node.js), Next.js
- **Java:** Spring Boot
- **PHP:** Laravel
- **Ruby:** Ruby on Rails

# Why Use a Web Framework?

- **Productivity:** Prebuilt tools and libraries save time.
- **Security:** Built-in protections against common web vulnerabilities (like XSS, CSRF).
- **Scalability:** Frameworks support structured, scalable architectures (e.g., MVC).
- **Maintainability:** Clean code organization.
- **Community Support:** Documentation, plugins, and active communities.

# Flask Framework

- Flask is a Python framework available under the BSD license.
- It is Inspired by the Sinatra Ruby framework.
- The Werkzeug WSGI toolkit and Jinja2 template are utilized by Flask.
- Werkzeug is a comprehensive Python library used to build WSGI (Web Server Gateway Interface) compliant web applications.

# Flask Framework

- It provides a wide range of tools and utilities, including **request** and **response** handling, URL **routing**, HTTP utilities, debugging, and more.
- Werkzeug is the foundation upon which many popular Python web frameworks like Flask are built.
- Flask is better suited for tiny and simple projects than Django.
- Hence, you can anticipate web server development, Google App Engine support, and integrated unit testing.

# Features

- **Routing:** URL to function mapping
- **Templating:** HTML generation with placeholders
- **Middleware:** Process requests/responses globally
- **ORM (Object Relational Mapping):** Map database tables to classes
- **Session and Cookies:** Handle user state and authentication
- **REST API Support:** JSON responses and HTTP methods



# Advantages

- Built-in development server with debugger.
- RESTful request dispatching.
- Integrated support for unit testing (code with quality).
- Uses Jinja2 templating (tags, filters, macros, and more).
- 100% WSGI 1.0 compatible.
- Several community-developed extensions provide the incorporation of new functionality.

# Limitations

- This framework can only be handled by highly qualified Flask developers, as it needs greater attention to detail and a deeper understanding of their relationships.

# Setting up a Flask environment

- To set up a Flask development environment, you'll need to **install Python**, a Python package manager like **pip**, and then use pip to **install Flask**.
- You'll also want to create a virtual environment to isolate your project's dependencies.
- To Create your own virtual environment

`virtualenv <env_name>>`

OR

`python -m venv <<myenv>>`

# Setting up a Flask environment

- Activate Virtual Environment on Windows

```
cd <env_name>
```

```
Scripts\activate
```

- Deactivate Virtual Environment

```
deactivate
```

***(Note: Remember to activate the relevant virtual environment every time you work on the project)***

# Setting up a Flask environment

- With the virtual environment activated, use pip to install flask
- Install Flask:

`pip install flask`

```
(.myenv) C:\Users\sunil\Desktop\My Project\Python\web>pip install flask
Collecting flask
  Downloading flask-3.1.1-py3-none-any.whl.metadata (3.0 kB)
Collecting blinker>=1.9.0 (from flask)
  Using cached blinker-1.9.0-py3-none-any.whl.metadata (1.6 kB)
Collecting click>=8.1.3 (from flask)
  Downloading click-8.2.0-py3-none-any.whl.metadata (2.5 kB)
Collecting itsdangerous>=2.2.0 (from flask)
```

# Setting up a Flask environment

- You can check the flask framework installed in your environment

pip list

```
(.myenv) C:\Users\sunil\Desktop\My Project\Python\web>pip list
```

| Package      | Version |
|--------------|---------|
| -----        | -----   |
| blinker      | 1.9.0   |
| click        | 8.2.0   |
| colorama     | 0.4.6   |
| Flask        | 3.1.1   |
| itsdangerous | 2.2.0   |
| Jinja2       | 3.1.6   |
| MarkupSafe   | 3.0.2   |
| pip          | 25.1.1  |
| Werkzeug     | 3.1.3   |

# Setting up a Flask environment

- **blinker 1.9.0** – Provides a fast dispatching system for signals, used internally by Flask (e.g., for events).
- **click 8.2.0** – A package used for creating command-line interfaces; Flask uses it for CLI commands.
- **colorama 0.4.6** – Makes ANSI escape character sequences (for color formatting) work on Windows.
- **Flask 3.1.1** – The main micro web framework you're likely using to build your application.

# Setting up a Flask environment

- **itsdangerous 2.2.0** – Used by Flask to handle safe data serialization (e.g., sessions, token generation).
- **Jinja2 3.1.6** – Flask's templating engine, used to generate HTML dynamically.
- **MarkupSafe 3.0.2** – Ensures safe handling of strings in Jinja2 templates (protects against injection).
- **pip 25.1.1** – Python's package installer.
- **Werkzeug 3.1.3** – A WSGI utility library Flask uses for request/response handling.



# Setting up a Flask environment

- To generate requirements.txt file

```
pip freeze > requirements.txt
```

- To install the dependencies later

```
pip install -r requirements.txt
```

# Setting up a Flask environment

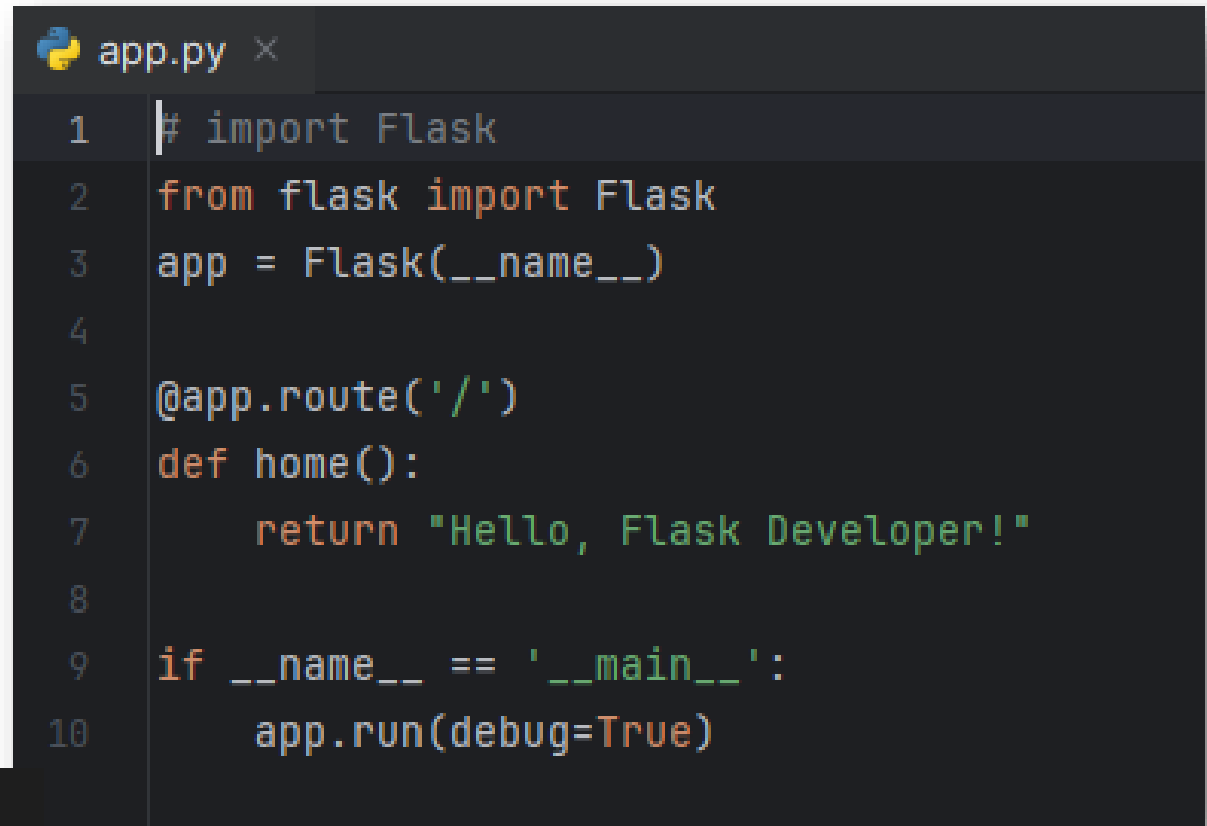
- Create a basic Flask application

```
# import Flask
from flask import Flask
app = Flask(__name__)

@app.route('/')
def home():
    return "Hello, Flask Developer!"

if __name__ == '__main__':
    app.run(debug=True)
```

```
10 # run the server
11 app.run(host="0.0.0.0", port=50100, debug=True)
```

A screenshot of a code editor window titled 'app.py'. The code is written in Python and matches the text shown in the previous blocks. The editor has a dark background with light-colored text. The code is as follows:

```
1 # import Flask
2 from flask import Flask
3 app = Flask(__name__)
4
5 @app.route('/')
6 def home():
7     return "Hello, Flask Developer!"
8
9 if __name__ == '__main__':
10     app.run(debug=True)
```

# Setting up a Flask environment

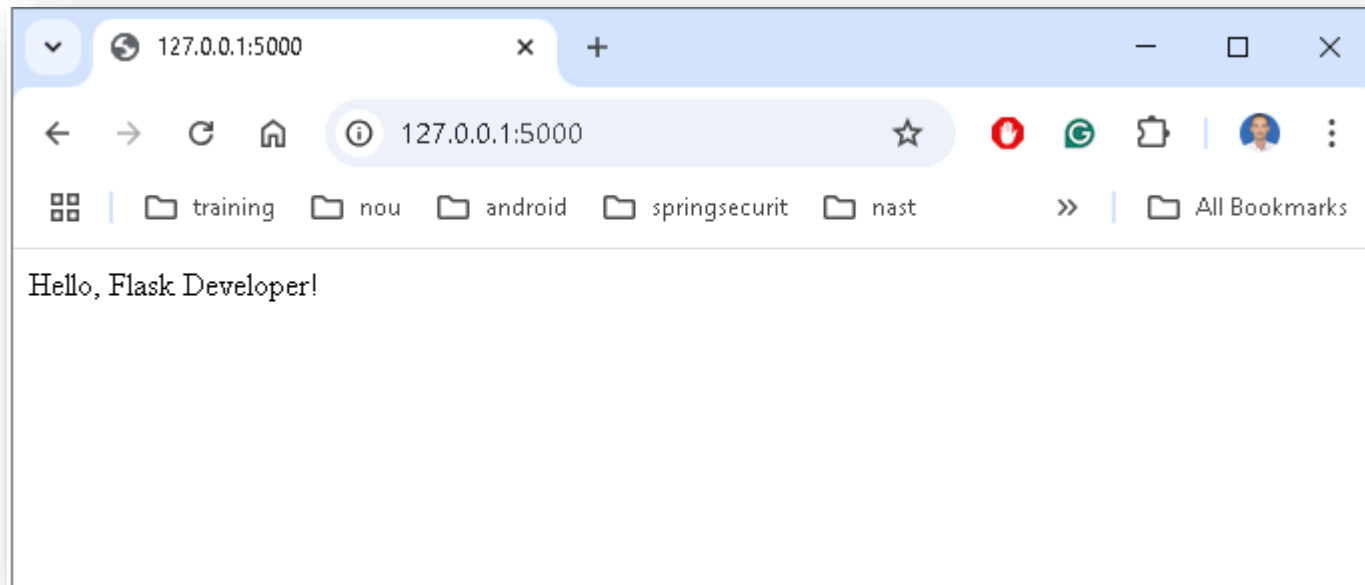
- Save file as app.py
- To run:

`python app.py`

```
(.myenv) C:\Users\sunil\Desktop\My Project\Python\web>python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 123-065-423
127.0.0.1 - - [17/May/2025 11:42:27] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [17/May/2025 11:42:27] "GET /favicon.ico HTTP/1.1" 404 -
* Detected change in 'C:\\Users\\sunil\\Desktop\\My Project\\Python\\web\\app.py', reloading
* Restarting with stat
* Debugger is active!
* Debugger PIN: 123-065-423
```

# Setting up a Flask environment

- Open any web Browser and type: `http://127.0.0.1:5000/`



# Best Practices

- Use virtual environments (e.g., venv or pipenv)
- Structure code (e.g., separate routes, templates, models)
- Follow RESTful principles for APIs
- Keep dependencies updated
- Secure inputs (validate user data, sanitize outputs)

# Understand Basic Flask Directory Structure

```
my_flask_app/
|
├── app/
|   ├── __init__.py      # Initializes the Flask app
|   ├── routes.py        # Application routes
|   ├── models.py        # Database models (optional)
|   ├── static/          # CSS, JS, images
|   └── templates/       # HTML templates (Jinja2)
|       └── index.html
|
├── venv/                # Virtual environment (optional)
├── run.py               # Entry point to run the app
└── requirements.txt     # List of packages
```

# Understand Basic Flask Directory Structure

- app/: Main application package
  - \_\_init\_\_.py: Creates and configures the Flask app
  - routes.py: Contains route handlers like @app.route("/")
  - templates/: HTML files using Jinja2 templating
  - static/: Static files like CSS, JS, images
- run.py: Starts the server
- venv/: Isolated Python environment
- requirements.txt: Can be created using `pip freeze > requirements.txt`

# Flask basics

- Routes
- Views, and
- Templates



# Routes

- In Python web frameworks, routes define how an application responds to specific URL requests.
- They map URLs to corresponding functions, which handle the logic for that particular endpoint.
- This system enables the creation of organized and maintainable web applications by associating specific actions with different URLs.
- Routes are typically defined using decorators provided by the web framework.
- For instance, in Flask, the `@app.route()` decorator is used to associate a function with a URL path.

# Routes Syntax

```
@app.route('/path', methods=['GET', 'POST'])  
def function_name():  
    return 'Response content'
```

Type: <http://localhost:5000/path> in web browser

# Basic Route

```
from flask import Flask  
app = Flask(__name__)
```

```
@app.route('/')  
def home():  
    return 'Welcome to the Home Page!'
```

Type: <http://localhost:5000> in web browser

# Route with Variable

```
@app.route('/hello/<name>')  
def hello(name):  
    return f'Hello, {name}!'
```

Type: <http://localhost:5000/hello/Sunil> in web browser

# Typed Route Variables

```
@app.route('/user/<int:user_id>')  
def user_profile(user_id):  
    return f'User Id: {user_id}'
```

Type: <http://localhost:5000/user/123> in web browser

# Routes with Methods

```
@app.route('/submit', methods=['GET', 'POST'])  
def submit():  
    if request.method == 'POST':  
        return 'Form Submitted'  
    return 'Submit Form Page'
```

Type: <http://localhost:5000/submit> in web browser

# Multiple Routes to the Same View

```
@app.route('/')  
@app.route('/index')  
def index():  
    return 'This is the index page'
```

Type: <http://localhost:5000/> or <http://localhost:5000/index> in browser

# Advanced: Blueprint Routes

```
# in routes.py
from flask import Blueprint

bp = Blueprint('main', __name__)

@bp.route('/')
def home():
    return 'Home via Blueprint'
```

```
# in __init__.py
from flask import Flask
from apps.routes import bp

def create_app():
    app = Flask(__name__)
    app.register_blueprint(bp)
    return app
```

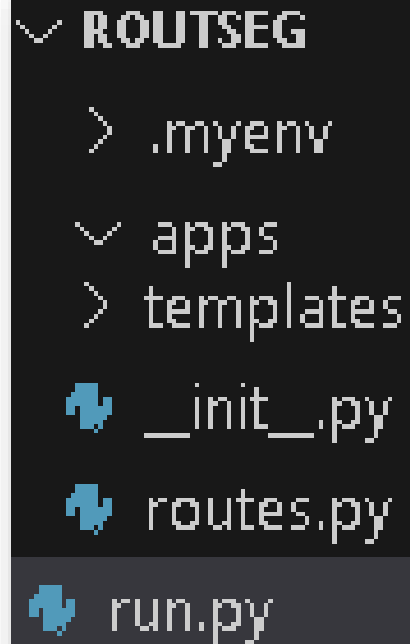


# Advanced: Blueprint Routes

```
from apps import create_app

# Create the Flask app using the
factory function
app = create_app()

if __name__ == '__main__':
    app.run(debug=True)
```



```
✓ ROUTSEG
  > .myenv
    ✓ apps
    > templates
      + __init__.py
      + routes.py
    + run.py
```

# Views

- A view is part of User Interface where user can interact with webpage or website.
- It is linked to a specific URL route.
- Processes the request.
- Returns a response (HTML, JSON, plain text, etc.)

## **For example,**

- When the user visits the /auth/register URL, the register view will return HTML with a form for them to fill out. When they submit the form, it will validate their input and either show the form again with an error message or create the new user and go to the login page.

# Basic View

```
from flask import Flask
```

```
app = Flask(__name__)
```

```
@app.route('/')  
def home():
```

```
    return "<h1>Welcome to the Homepage</h1>"
```

# View Returning HTML

```
from flask import render_template
```

```
@app.route('/about')
```

```
def about():
```

```
    return render_template('about.html')
```

# View Returning HTML

```
from flask import render_template
```

```
@app.route('/about')
```

```
def about():  
    return render_template('about.html')
```

```
templates/about.html
```

```
<!DOCTYPE html>  
<html>  
<head>  
    <title>About</title>  
</head>  
<body>  
    <h1>About Page</h1>  
    <p>This is a simple Flask  
application.</p>  
</body>  
</html>
```

# Templates

- Templates are HTML files used to create dynamic web pages.
- Flask uses Jinja2 as its templating engine.
- With templates, you can embed Python-like code (e.g., loops, conditions) inside your HTML.
- Jinja2 Syntax (inside templates)
  - `{{ variable }}` Display a variable
  - `{% ... %}` Logic: loops, conditions, etc.
  - `{# ... #}` Comments (not shown in output)

# Connecting Views and Templates

```
@app.route('/user/<username>')
def user_profile(username):
    return
    render_template('profile.html',
name=username)
```

```
templates/profile.html
<!DOCTYPE html>
<html>
<head>
    <title>Profile</title>
</head>
<body>
    <h1>Hello, {{ name }}!</h1>
</body>
</html>
```

# Passing Data from View to Template

```
@app.route('/programs')
def users():
    pro_list = ['BCA', 'BE Computer', 'BBA',
               'BE Civil']
    return
render_template('programs.html', items
               = pro_list)
```

```
<h2>NAST Programs</h2>
<ul>
    {% for p in items %}
        <li>{{ p }}</li>
    {% endfor %}
</ul>
```



# Variables & Conditions (if, elif, else)

- **Conditions**

```
{% if age >= 18 %}
```

```
    <p>You are an adult.</p>
```

```
{% elif age >= 13 %}
```

```
    <p>You are a teenager.</p>
```

```
{% else %}
```

```
    <p>You are a child.</p>
```

```
{% endif %}
```

- **Variables**

```
<p>Hello, {{ name }}</p>
```

# Loops (for)

<ul>

{% for user in users %}

    <li>{{ user }}</li>

{% endfor %}

</ul>

{% for user in users %}

    {{ loop.index }}. {{ user }}

{% endfor %}

## **Note:**

loop.index    1-based index

loop.index0   0-based index

loop.last     True if last item

# Filters

{{ name|upper }} → ALICE

{{ name|lower }} → alice

{{ name|capitalize }} → Alice

{{ numbers|length }} → 3

{{ content|safe }}

→ Renders HTML without escaping

## Filter

- | Filter         | Description                   |
|----------------|-------------------------------|
| • upper        | Convert to uppercase          |
| • lower        | Convert to lowercase          |
| • capitalize   | First letter uppercase        |
| • length       | Length of a list or string    |
| • safe         | Renders raw HTML              |
| • replace(a,b) | Replace a with b              |
| • default('x') | Use 'x' if value is undefined |

# Macros (Reusable Code Blocks)

```
{% macro input(name, value="", type='text') %}  
    <input type="{{ type }}" name="{{ name }}" value="{{ value }}">  
{% endmacro %}
```

```
{{ input('username') }}
```

# Template Inheritance

## **base.html**

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
    <title>{% block title %}My App{% endblock %}</title>
```

```
</head>
```

```
<body>
```

```
    {% block content %}{% endblock %}
```

```
</body>
```

```
</html>
```

# Template Inheritance

## **child.html**

```
{% extends 'base.html' %}
```

```
{% block title %}Home Page{% endblock %}
```

```
{% block content %}
```

```
    <h1>Welcome to the Homepage</h1>
```

```
{% endblock %}
```

# Other Use cases

- **Include**

```
{% include 'navbar.html' %}
```

- **Comments**

```
{# This is a comment and will not be rendered #}
```

- **Set Variables**

```
{% set greeting = "Hello" %}
```

```
<p>{{ greeting }}, World!</p>
```

# Other Syntaxes

- **Whitespace Control**

- You can control whitespace using a hyphen (-):

- `{ name -}`

- `{{- name }}`

- `{{- name -}}`

- These remove whitespace before and/or after the variable.



# Handling HTTP requests and responses

- Handling HTTP requests and responses in a **Python Flask web application** is straightforward and efficient due to Flask's minimalist design.
- In Flask, each route (endpoint) is tied to a Python function that handles an HTTP request and returns a response.
- Incoming request data can be accessed using the request object from the flask module.
- Flask automatically handles much of the response construction. Return values from view functions are automatically converted into Response objects.

# Handling HTTP requests and responses

## Basic Structure

- We need to import **request** & **make\_response** modules

For example,

```
from flask import Flask, request, jsonify, make_response  
app = Flask(__name__)
```

# Handling HTTP Requests

- **GET Request – Reading URL parameters**

```
@app.route('/hello', methods=['GET'])
```

```
def hello():
```

```
    name = request.args.get('name', 'Guest')
```

```
    return f'Hello, {name}!'
```

`request.args.get('name')`: Gets a query parameter like  
`/hello?name=Sunil`

# Handling HTTP Requests

- **POST Request – Reading JSON body**

```
@app.route('/submit', methods=['POST'])
```

```
def submit():
```

```
    data = request.get_json()
```

```
    name = data.get('name', 'Guest')
```

```
    return jsonify({'message': f'Data received from {name}'})
```

`request.get_json()`: Parses JSON sent in the body of the request.

# Handling HTTP Requests

- **Form Data – eg., from HTML forms**

```
@app.route('/form', methods=['POST'])
```

```
def form_submit():
```

```
    name = request.form.get('name')
```

```
    return f'Form submitted by {name}'
```

request.form: Accesses form fields sent with application/x-www-form-urlencoded.

# Handling HTTP Requests

- **File Uploads**

```
@app.route('/upload', methods=['POST'])  
def upload_file():  
    file = request.files['file']  
    file.save(f"./uploads/{file.filename}")  
    return 'File uploaded successfully'
```

# **Sending HTTP Responses**

## **Simple Text Response**

```
@app.route('/text')  
def text_response():  
    return "This is a plain text response"
```

## **JSON Response**

```
@app.route('/json')  
def json_response():  
    return jsonify({'status': 'success', 'data': [1, 2, 3]})
```

# Sending HTTP Responses

- **Custom Response with Status Code**

```
@app.route('/custom')
```

```
def custom_response():
```

```
    return make_response(jsonify({'error': 'Not Found'}), 404)
```



# Setting request, response Header

```
@app.route('/headers')  
def headers():  
    user_agent = request.headers.get('User-Agent')  
    return f'Your user agent is: {user_agent}'
```

---

```
@app.route('/set-header')  
def set_header():  
    response = make_response("Headers set")  
    response.headers['X-Custom-Header'] = 'MyHeaderValue'  
    return response
```

# Building Basic Web Functionality

- **Create a Greeting Flask App**

```
from flask import Flask,  
render_template, request, redirect,  
url_for
```

```
app = Flask(__name__)
```

```
# Home route
```

```
@app.route('/')  
def index():
```

```
    return render_template('index.html')
```

```
# Handle form submission
```

```
@app.route('/submit',  
methods=['POST'])
```

```
def submit():
```

```
    name = request.form.get('name')
```

```
    return f"Hello, {name}!"
```

```
if __name__ == '__main__':
```

```
    app.run(debug=True)
```

# Building Basic Web Functionality

- **Place an index.html file in the template/ folder:**

```
<!DOCTYPE html>
<html>
<head> <title>Flask Basic Web</title> </head>
<body>
  <h1>Enter Your Name</h1>
  <form action="{{ url_for('submit') }}" method="POST">
    <input type="text" name="name" required>
    <input type="submit" value="Submit">
  </form>
</body>
</html>
```

# Building Basic Web Functionality

- **Place a style.css file in the static/ folder:**

```
body {  
    font-family: Arial, sans-serif;  
    background-color: #f9f9f9;  
    padding: 20px;  
}
```

- **Link it in your HTML:**

```
<head>  
    <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">  
</head>
```

# Building Basic Web Functionality

- Implementing form submission and input validation
- Understanding and managing cookies and sessions

# Cookies

- Cookies are small pieces of data stored in the user's browser and sent with every request to the server.
- The browser then stores these cookies and sends them back to the server with every subsequent request to the same domain.
- Flask allows you to set and retrieve cookies easily using the request and response objects.
- They are primarily used for:
  - **Session Management:** Keeping users logged in, remembering items in a shopping cart.
  - **Personalization:** Storing user preferences (e.g., language, theme).
  - **Tracking:** Website analytics and user behavior tracking.

# Cookies

- You'll typically create a Response object using `make_response()` or get it from a `render_template()` call, and then use its `set_cookie()` method.

```
1  from flask import Flask, make_response, request, render_template
2
3  app = Flask(__name__)
4
5  @app.route('/set_cookie')
6  def set_cookie():
7      #resp= make_response("Cookie Information Set")
8      resp = make_response(render_template('index.html')) # from template
9      resp.set_cookie('username', 'sunil', max_age=60*60*24*2) # Expires in 2 days
10     return resp
11
```

# Cookies

- **set\_cookie()** parameters
- **key**: The name of the cookie.
- **value**: The value to store in the cookie.
- **max\_age**:
  - The maximum age of the cookie in seconds.
  - If not set, the cookie is deleted when the browser closes.
- **expires**: A datetime object indicating when the cookie should expire.
- **path**: The path for which the cookie is valid. Defaults to /.



# Cookies

- **domain:** The domain for which the cookie is valid.
- **secure:**
  - If True, the cookie will only be sent over HTTPS.
  - Highly recommended for sensitive data.
- **httponly:**
  - If True, the cookie cannot be accessed by client-side JavaScript.
  - Highly recommended to prevent XSS attacks.
- **samesite:**
  - Controls when cookies are sent with cross-site requests ('Lax' or 'Strict').
  - 'Lax' is often a good default.

# Cookies

```
12 @app.route('/get_cookie')
13 def get_cookie():
14     username = request.cookies.get('username')
15     if username:
16         return f"Welcome back, {username}!"
17     return "No username cookie found."
18
19 @app.route('/delete_cookie')
20 def delete_cookie():
21     resp = make_response("Cookie deleted!")
22     resp.delete_cookie('username')
23     #response.set_cookie('username', '', expires=0)
24     return resp
```

```
25
26 @app.route('/')
27 def index():
28     return render_template('index.html')
29
30 if __name__ == '__main__':
31     app.run(debug=True)
32
```

# Security Considerations for Cookies

- **Client-Side Storage:** Cookies are stored on the user's browser, which means they can be viewed and potentially tampered with by the user.
- **Sensitive Data:** Never store sensitive information (like passwords or unencrypted personal data) directly in cookies.
- **secure flag:** Always use `secure=True` when your application is served over HTTPS to prevent cookies from being sent over insecure connections.
- **httponly flag:** Use `httponly=True` to prevent JavaScript from accessing the cookie, mitigating certain Cross-Site Scripting (XSS) attacks.

# Session

- Sessions allow you to store user-specific data server-side (or client-side with signed cookies) across multiple requests.
- Flask uses signed cookies by default to store session data securely.
- This session ID then allows the server to retrieve the associated session data.
- Before you can use sessions, you must set a `SECRET_KEY` for your Flask application.
- This key is used to sign the session cookie, making it tamper-proof. Keep this key secret and generate a strong, random one.

# Session

- By default, Flask sessions are temporary and expire when the browser is closed.
- To make a session permanent (persisting for a defined duration), you need to set `session.permanent = True` and configure `PERMANENT_SESSION_LIFETIME`.

# Key Points

- Sessions are built on top of cookies but are managed server-side or as signed cookies.
- Flask uses the session object, which acts like a dictionary.
- Session data is signed using a secret key to prevent tampering.
- By default, session data is stored in a cookie, but you can use extensions like Flask-Session for server-side storage.

# Session Example

```
1  from flask import Flask, session, redirect, url_for, request, render_template
2  import os
3  from datetime import timedelta
4
5  app = Flask(__name__)
6  app.secret_key = os.urandom(24) # Generate a strong, random key
7  # or: app.config['SECRET_KEY'] = 'your_super_secret_key_here'
8  app.config['PERMANENT_SESSION_LIFETIME'] = timedelta(minutes=30) # Session lasts 30 minutes
9
10 @app.route('/login', methods=['GET', 'POST'])
11 def login():
12     if request.method == 'POST':
13         username = request.form['username']
14         session['username'] = username
15         #session.permanent = True
16         #To Make the session permanent
17         return redirect(url_for('dashboard'))
18     return render_template('login.html')
19
```

# Session Example

```
20 @app.route('/dashboard')
21 def dashboard():
22     if 'username' in session:
23         username = session['username']
24         return f"Hello, {username}! <a href='/logout'>Logout</a>"
25     return redirect(url_for('login'))
26
27 @app.route('/logout')
28 def logout():
29     session.pop('username', None) # Remove 'username' from session
30     # or session.clear() to remove all items
31     return redirect(url_for('login'))
32
33 if __name__ == '__main__':
34     app.run(debug=True)
```



# Flask redirect

- In Flask, the `redirect()` function is used to redirect the user to a different URL.
- It can be used to redirect to other routes within the same application or to external websites.
- To use `redirect()`, it must first be imported from the flask module.
- The function takes a single argument, which is the URL to redirect to.
- This URL can be a relative path, an absolute path, or a fully qualified URL.
- Here's an example of how to use `redirect()` to redirect to another route within the same application:

# Flask redirect

```
from flask import Flask, redirect, url_for
```

```
app = Flask(__name__)
```

```
@app.route('/')  
def index():  
    return 'This is the index page.'
```

```
@app.route('/admin')  
def admin():  
    return redirect(url_for('index'))
```

```
if __name__ == '__main__':  
    app.run(debug=True)
```

# Flask redirect

- In the above example, when the user visits the /admin route, they will be automatically redirected to the / route.
- The url\_for() function is used to generate the URL for the / route.
- This ensures that the redirect will still work even if the route's URL is changed.
- It is also possible to redirect to an external website using redirect():

# Flask redirect

```
from flask import Flask, redirect  
app = Flask(__name__)
```

```
@app.route('/')  
def index():  
    return redirect('https://www.nast.edu.np')
```

```
if __name__ == '__main__':  
    app.run(debug=True)
```

- In this example, when the user visits the / route, they will be redirected to <https://www.nast.edu.np>

# Message flashing

- Message flashing in Flask provides a way to display one-time notification messages to users.
- These messages are often used to provide feedback after an action, such as a successful form submission or an error during login.
- Flask's flashing system stores messages in the session and retrieves them for display on the next request.
- To use message flashing you need to Import the flash function from the flask module.

# Message flashing

```
from flask import Flask, flash, render_template, request, redirect,  
url_for
```

- Use flash() to add messages:
  - Call the `flash()` function with the message text and optionally a category (e.g., `'info'`, `'error'`, `'success'`).
- Retrieve and display messages:
  - In your templates, use the `get_flashed_messages()` function to retrieve and display flashed messages.

# Message flashing

```
@app.route('/login', methods=['POST'])
def login():
    username = request.form['username']
    password = request.form['password']
    if username == 'admin' and password == 'password':
        flash('Login successful!', 'success')
        return redirect(url_for('dashboard'))
    else:
        flash('Incorrect username or password.', 'error')
        return render_template('login.html')
```

# Message flashing

```
{% with messages = get_flashed_messages(with_categories=true) %}  
  {% if messages %}  
    {% for category, message in messages %}  
      <div class="alert alert-{{ category }}">  
        {{ message }}  
      </div>  
    {% endfor %}  
  {% endif %}  
{% endwith %}
```



# Message flashing

- Configure secret\_key:
  - Ensure your Flask application has a secret\_key configured to enable session usage for flashing.

```
app = Flask(__name__)
```

```
app.config['SECRET_KEY'] = 'your_secret_key'
```

# File upload

- File uploading is a typical task in web apps.
- Taking care of file upload in Flask is simple all we need is to have an HTML form with the enctype set to **multipart/form** information to publish the file into the URL.
- The server-side flask script brings the file from the request object utilizing the **request.files[]** Object.

# File upload

## index.html

```
<html>
<head>
  <title>File Uploads</title>
</head>
<body>
  <form action = "/success" method = "post"
enctype="multipart/form-data">
    <input type="file" name="file" />
    <input type = "submit" value="Upload">
  </form>
</body>
</html>
```

## ack.html

```
<html>
  <head>
    <title>Information</title>
  </head>
  <body>
    <p>File uploaded successfully</p>
    <p>File Name: {{name}}</p>
  </body>
</html>
```

# File upload

```
from fileinput import filename
from flask import Flask, render_template, request
import os
```

```
app = Flask(__name__)
app.config['UPLOAD_FOLDER'] = './uploads'
app.config['MAX_CONTENT_LENGTH'] = 16 * 1024 * 1024 # Limit size to 16 MB
```

```
@app.route('/')
def main():
    return render_template("index.html")
```

# File upload

```
@app.route('/success', methods = ['POST'])
def success():
    if request.method == 'POST':
        f = request.files['file']
        path = os.path.join(app.config['UPLOAD_FOLDER'], f.filename)
        f.save(path)
        return render_template("ack.html", name = f.filename)

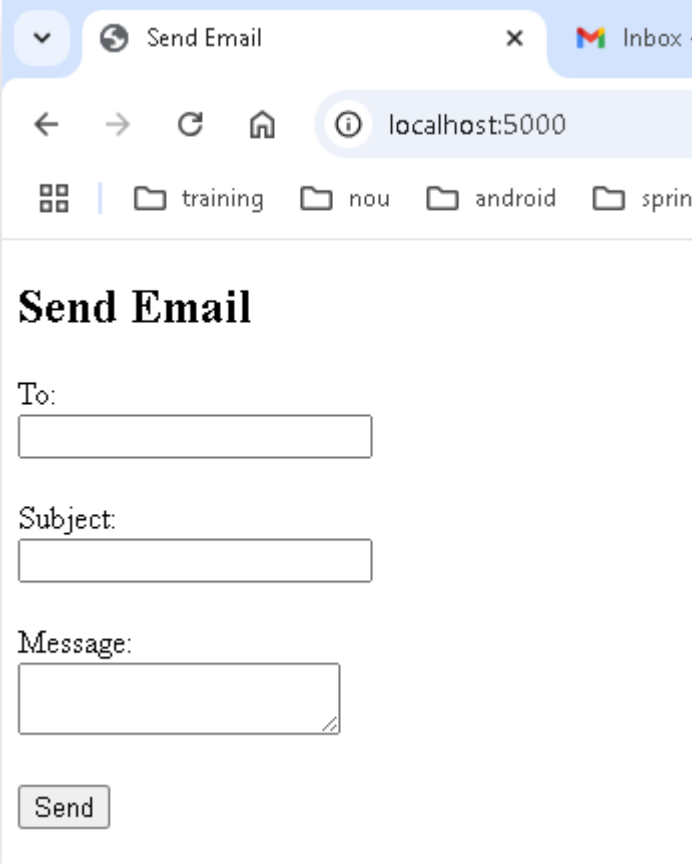
if __name__ == '__main__':
    app.run(debug=True)
```

# Sending Email

- Python, being a powerful language don't need any external library to import and offers a native library to send emails- "SMTP lib".  
"smtplib" creates a Simple Mail Transfer Protocol client session object which is used to send emails to any valid email id on the internet.
- Sending email in a Flask application is typically done using an external mail server (like Gmail, SendGrid, etc.) along with a package like Flask-Mail or Flask-Mailman.
- Below is a basic example using Flask-Mail: `pip install Flask-Mail`

# Sending Email

```
<!DOCTYPE html>
<html>
<head>
  <title>Send Email</title>
</head>
<body>
  <h2>Send Email</h2>
  <form method="POST" action="/send_email">
    <label>To:</label><br>
    <input type="email" name="email" required><br><br>
    <label>Subject:</label><br>
    <input type="text" name="subject" required><br><br>
    <label>Message:</label><br>
```



A screenshot of a web browser window displaying a 'Send Email' form. The browser's address bar shows 'localhost:5000'. The form has a title 'Send Email' and three input fields: 'To:', 'Subject:', and 'Message:'. The 'To:' field is a text input, 'Subject:' is a text input, and 'Message:' is a text area. A 'Send' button is located at the bottom of the form.

# Sending Email

```
<textarea name="message" required></textarea><br><br>
<button type="submit">Send</button>
</form>
{% with messages = get_flashed_messages(with_categories=true) %}
  {% if messages %}
    <ul>
      {% for category, message in messages %}
        <li><strong>{{ category.title() }}:</strong> {{ message }}</li>
      {% endfor %}
    </ul>
  {% endif %}
{% endwith %}
</body>
</html>
```



# Sending Email

```
from flask import Flask, render_template, request, redirect, url_for, flash
from flask_mail import Mail, Message
```

```
app = Flask(__name__)
app.secret_key = 'your_secret_key'
```

```
# Email configuration
```

```
app.config['MAIL_SERVER'] = 'smtp.gmail.com'
app.config['MAIL_PORT'] = 587
app.config['MAIL_USE_TLS'] = True
app.config['MAIL_USERNAME'] = 'username@gmail.com'
app.config['MAIL_PASSWORD'] = 'app_password'
```

```
mail = Mail(app)
```

# Sending Email

```
@app.route('/')  
def index():  
    return render_template('send-email.html')  
  
@app.route('/send_email', methods=['POST'])  
def send_email():  
    try:  
        email = request.form['email']  
        subject = request.form['subject']  
        message = request.form['message']
```

# Sending Email

```
msg = Message(subject, sender=app.config['MAIL_USERNAME'], recipients=[email])
msg.body = message
mail.send(msg)
```

```
    flash('Email sent successfully!', 'success')
except Exception as e:
    flash(f'Error sending email: {str(e)}', 'danger')
```

```
return redirect(url_for('index'))
```

```
if __name__ == '__main__':
    app.run(debug=True)
```

# Next Chapter